Práctica 5

Kiosco multimedia

Alumno: Vázquez Apolonio Armando

Materia: Fundamentos de Sistemas Embebidos

1. Introducción

En esta práctica se utilizo una Raspberry PI como sistema embebido para implementarlo como tipo kiosco multimedia con la capacidad de reproducir archivos de audio,video e imágenes de forma automática.Para ello se configuro un entorno ligero de ejecución y se emplearon herramientas como FFmpeg,VLC media player y NoDM, con el fin de ejecutar contenido multimedia sin depender del escritorio de sistema operativos tradicionales

2. Antecedentes

De acuerdo con SSL Digital, los kioscos digitales son terminales que permiten mostrar información o brindar servicios de manera autónomoSSL Digital, 2024

3. Desarrollo

1.[2 pts] Modifique el archivo de ejemplo minikiosk.py para que reproduzca el video de prueba durante 10 segundos e inmediatamente después reproduzca en un bucle infinito cada una de las imágenes de muestra por espacio de 3 segundos. código:

```
# 1. Video inicial (10 seg)
video = inst.media_new(video_file)
player.set_media(video)
player.play()
time.sleep(10)
player.stop()
# 2. Bucle inteligente
while True:
    if os.path.exists(usb_pics) and len(os.listdir(usb_pics)) > 0:
        actual_path = usb_pics
        print("Reproduciendo desde USB...")
    else:
        actual_path = local_pics
        print("USB no detectada. Usando fotos locales...")
    images = [f for f in os.listdir(actual_path) if f.endswith(('.jpg', '.png'))]
    for img_name in images:
        img_full_path = os.path.join(actual_path, img_name)
        img_media = inst.media_new(img_full_path)
        player.set_media(img_media)
        player.play()
        time.sleep(3)
```

Se modificó el código para que el sistema verificara si existe una memoria USB con imágenes disponibles y, en caso afirmativo, las reprodujera en lugar de las almacenadas localmente.

2. [4 pts] Integre el archivo minikiosk.py modificado con el detector de medios extraíbles usbdetect.py para que la presentación infinita de imágenes se detenga cuando se inserta una USB y, en lugar de las imágenes de muestra, se reproduzcan las imágenes del medio extraíble.

```
import glob
while True:
```

```
    dispositivos = glob.glob('/dev/sd[a-z][1-9]')
    if dispositivos:
        dispositivo_actual = dispositivos[0]
        os.system(f'sudo mount {dispositivo_actual} /mnt/usb 2>/dev/null')
    if os.path.exists(usb_pics) and len(os.listdir(usb_pics)) > 0:
        actual_path = usb_pics
        print(f"USB detectada en {dispositivo_actual}. Reproduciendo desde memoria.")
    else:
        actual_path = local_pics
        print("USB no detectada. Usando fotos locales.")
    images = [f for f in os.listdir(actual_path) if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    images.sort()
    for img_name in images:
        img_full_path = "file://" + os.path.join(actual_path, img_name)
        img_media = inst.media_new(img_full_path)
        player.set_media(img_media)
        player.play()
        print(f"Mostrando: {img_name}")
        time.sleep(3)
```

Se añadió la detección automática de dispositivos USB mediante glob, así como el montaje de la memoria en /mnt/usb. Además, se ajustó la carga de imágenes para aceptar archivos .jpg, .jpeg y .png, ordenándolos antes de su reproducción.

3. [+2 pts] Modifique el programa anterior para que el video se reproduzca por espacio de 20 segundos. El video debe comenzar con volumen cero que se incrementará gradualmente hasta 100

```
# Configuración de VLC con salida de audio por jack 3.5 mm
inst = vlc.Instance(
    '--fullscreen','--no-osd','--mouse-hide-timeout=0','--alsa-audio-device=hw:0,0'
)
#  Video con volumen gradual
print("Iniciando video: 20 segundos con efectos de volumen...")
video = inst.media_new(video_file)
player.set_media(video)
player.play()
time.sleep(1)
# Fade-in
print("Iniciando Fade-in...")
for v in range(0, 101, 20):
    player.audio_set_volume(v)
    print(f"Volumen subiendo: {v}%")
    time.sleep(1)
print(" Volumen al máximo (100%)...")
player.audio_set_volume(100)
time.sleep(10)
print("Iniciando Fade-out...")
for v in range(100, -1, -20):
    player.audio_set_volume(v)
    print(f"Volumen bajando: {v}%")
    time.sleep(1)
player.stop()
```

Se modificó la configuración de VLC para forzar la salida de audio por el conector de 3.5 mm. Además, se agregó un control gradual del volumen durante la reproducción del video, implementando un efecto de fade-in, un periodo de volumen máximo y finalmente un fade-out.

4. Cuestionario

[1.0pt] ¿Qué funciones o métodos se utilizan para subir y bajar el volumen durante una reproducción? Ilustrecon código de ejemplo. Para subir y bajar el volumen durante la reproducción en python-vlc se utiliza el método `audio_set_volume(valor)` del objeto `MediaPlayer`, el cual permite establecer el nivel de volumen. Para obtener el volumen actual se emplea el método `audio_get_volume()`. El aumento o disminución del volumen se realiza modificando el valor actual.

```
import vlc
player = vlc.MediaPlayer('/home/apolo/videos/video.mp4')
player.play()# Crear reproductor
vol = player.audio_get_volume()# Obtener volumen actual
player.audio_set_volume(vol + 20)# Subir volumen
player.audio_set_volume(vol - 30)# Bajar volumen
print("Volumen:", player.audio_get_volume()) # Mostrar volumen actual
```

2. [1.0pt] ¿Qué funciones o métodos se utilizan para ejecutar la reproducción en modo de pantalla completa? Ilustre con código de ejemplo. Para ejecutar la reproducción en modo de pantalla completa en python-vlc se pueden emplear dos enfoques el primero consiste en crear la instancia de VLC utilizando el parámetro `--fullscreen`,lo que inicia la reproducción directamente en pantalla completa ,el segundo consiste en utilizar el método `set_fullscreen(True)` del objeto `MediaPlayer`, el cual permite activar este modo durante la ejecución.

```
import vlc
inst = vlc.Instance('--fullscreen')
player = inst.media_player_new()
media =inst.media_new('/home/apolo/videos/video.mp4')
player.set_media(media)
player.play()
player.set_fullscreen(True)
```

3. [2.0pt] ¿Cuáles son las funciones que permiten desplazarse en una lista de reproducción y controlarla (siguiente, anterior, pausa, detener y reanudar)? Ilustre con código de ejemplo

Para controlar una lista de reproducción en `python-vlc` se emplea el objeto `MediaListPlayer`, el cual dispone de métodos como `next()` para avanzar al siguiente elemento, `previous()` para regresar al anterior, `play()` para iniciar la reproducción, `pause()` o `set_pause()` para pausar o reanudar, y `stop()` para detener la lista.

```
#Crear lista de medios
media_list = vlc.MediaList([
"/home/apolo/pictures/img1.jpg","/home/apolo/pictures/img2.jpg","/home/apolo/pictures/img3.jpg"
])
list_player = inst.media_list_player_new()# Crear reproductor de lista
player = inst.media_player_new()
list_player.set_media_player(player)
list_player.set_media_list(media_list)
list_player.play()        # Iniciar
list_player.next()        # Siguiente
list_player.previous()    # Anterior
list_player.pause()       # Pausar
list_player.play()        # Reanudar
list_player.stop()        # Detener
```

5. Conclusiones

En esta práctica se logró configurar una Raspberry Pi como un kiosco multimedia capaz de reproducir contenido de forma automática, integrando herramientas de software para la visualización de video e imágenes. Con ello, se comprendió mejor cómo un sistema embebido puede destinarse a una función específica de reproducción multimedia. Durante el desarrollo se presentaron dificultades en la salida de audio, ya que fue necesario ajustar su configuración para que funcionara correctamente. También hubo problemas en la parte de reproducir el video al introducir la

memoria USB, debido a la detección y montaje del dispositivo. A pesar de ello, la práctica permitió reforzar la integración entre hardware, software y automatización en la Raspberry Pi.

Evidencias en videos : https://drive.google.com/drive/folders/1FoyNKYKlwzuCL1bsvg3PdblX0Bt-lkuR?usp=drive_link

Referencias

SSL Digital. (2024, noviembre). *¿Qué son los kioscos digitales y cómo pueden ayudar a tu empresa?* [Consultado el 22 de marzo de 2026]. https://ssl.com.mx/detalle-blog/que-son-los-kioscos-digitales-y-como-pueden-ayudar-a-tu-empresa